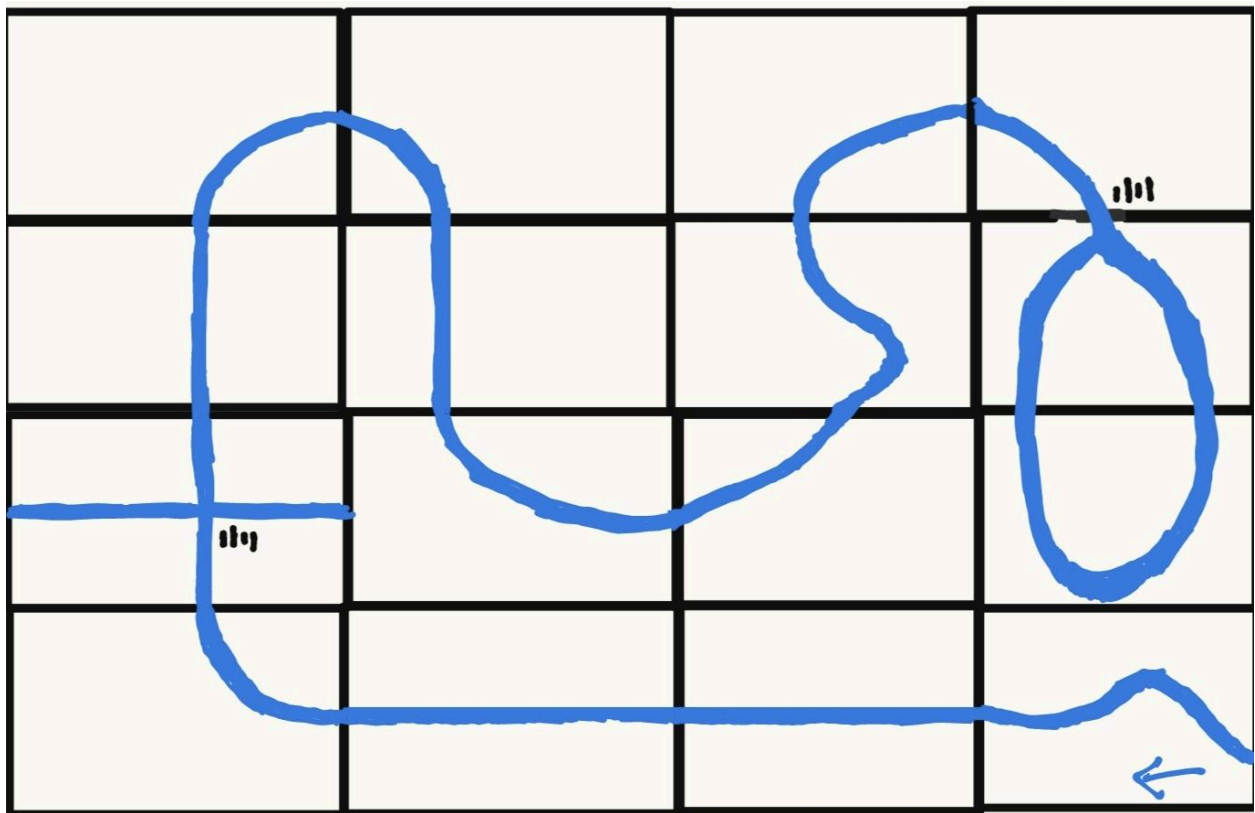


My Map



The map is designed on a 4×4 grid and begins from the bottom-right side. The route first follows a curved S-shaped track, then continues through two straight track sections. After that, the path turns right and reaches an intersection where the first barcode, Barcode 15, is placed. This barcode instructs the BB Car to continue moving straight through the intersection.

After passing the intersection, the car follows a single straight track before entering two curved U-shaped turns. The route then continues with another straight section, followed by another U-shaped loop. Next, the car moves through a second curved S-shaped path and another U-shaped loop before detecting the second barcode, Barcode 0, which instructs the car to make a left turn. Finally, the route ends with the car following a circular track.

Overall, the map consists of four straight track sections, two S-shaped curved sections, seven turning sections, and one circular track. This layout allows the BB Car to be tested on different navigation conditions, including straight-line movement, curved paths, intersections, barcode-based decisions, and circular tracking.

Challenges Faced During BB Car Navigation

During this project, one of the main challenges was making the BB Car navigate reliably in a real physical environment. The system had to combine Pixy2 line following, LaserPing obstacle detection, Feedback360 motor feedback, barcode/intersection handling, MQTT communication, and F769 remote control. Although each component could work individually, integrating them into one stable real-time navigation system was difficult because sensor noise, timing issues, network configuration, and control tuning all affected the car's movement.

The first major challenge was achieving a stable line following. The BB Car often swayed left and right instead of smoothly following the black line. This happened because the Pixy2 line error changed quickly, so the steering controller had to be tuned carefully. If the proportional and derivative gains were too strong, the car overcorrected and oscillated. If they were too weak, the car reacted too slowly, failed to follow curves, or lost the line. Speed, deadband, correction limits, and filtering also had to be adjusted so the car could respond smoothly without turning in circles.

Another challenge was integrating the LaserPing sensor for obstacle detection. At first, the sensor produced unreliable distance readings, including values that stayed fixed, changed suddenly, or falsely detected obstacles. This caused the car to enter obstacle handling mode at the wrong time. The sensor's mounting direction, environmental reflections, and measurement noise affected the readings, so the obstacle threshold and confirmation logic had to be tuned carefully to reduce false triggers.

Obstacle handling was originally planned as a dynamic avoidance behavior, where the car would detect an obstacle, move around it, and then return to the original line path. However, this approach became difficult to make reliable because the car could not always confirm whether it had fully passed the obstacle or whether it had successfully returned to the correct track. A fixed-time avoidance sequence could cause the car to lose the line, rejoin at the wrong angle, or enter a search state for too long. Because of this, the final decision was to simplify the behavior: when an obstacle is detected, the car stops or freezes until the obstacle is removed. This made the system safer, more predictable, and easier to demonstrate reliably.

Line reacquisition after obstacle avoidance was also a major issue during development. After attempting to move around an obstacle, the car was often no longer aligned with the original track. If the search behavior was too aggressive, the car could miss the line or re-enter at the wrong angle. If it was too slow, recovery took too long or the car became stuck in a search pattern. This was one of the reasons the final design prioritized obstacle stopping instead of dynamic avoidance.

The project also involved barcode-based or intersection-based decision making. The Pixy2 had to detect intersections or barcodes to decide whether the car should turn left, turn right, continue straight, or stop. This introduced additional difficulty because intersections could be missed or detected inconsistently. Debouncing and turn-command management were needed so the car would not make incorrect route decisions or repeat the same command at the wrong time.

Position tracking on the F769 map was another challenge. Initially, map updates based on intersections were not always accurate because the Pixy2 could miss intersection events. Feedback360 odometry was considered or used to estimate movement from wheel rotation, which could provide more continuous position tracking. However, this introduced issues such as feedback noise and drift. Even when the car was not moving, small feedback jitter could still cause the displayed distance to increase. Filtering and movement-condition checks were needed so odometry would only update when the car was actually supposed to move.

MQTT communication also created integration challenges. The BB Car had to publish status information such as navigation state, LaserPing distance, position, heading, and feedback distance while also receiving remote commands from the F769 board. If MQTT servicing was too infrequent or the connection was unstable, the car could appear connected through serial output while the status messages still did not appear on the subscriber terminal. The MQTT loop, publish timing, and payload size had to be adjusted so communication would not interfere with navigation.

Another issue was configuring the MQTT broker correctly. At one point, the Mosquitto broker appeared to run, but status messages were not received properly by external devices. This was caused by the broker running in local-only mode, preventing the BB Car and F769 board from connecting correctly over the network. The configuration had to be changed to allow remote access. It was also important to use different MQTT client IDs for the BB Car and F769 board so the broker would not disconnect one device when the other connected.

Manual override through the F769 remote added another layer of complexity. The system needed to separate automatic navigation mode from manual control mode. Commands such as forward, backward, left, right, stop, and auto had to be received through MQTT and translated into motor actions. At the same time, the car still needed to maintain safe behavior and publish status updates so the remote interface could display useful information.

Overall, the biggest challenge was not writing each individual function, but making all components work together reliably in real time. The Pixy2 camera, LaserPing sensor, Feedback360 servos, motor controller, obstacle detection logic, barcode/intersection detection, MQTT telemetry, and F769 remote interface all affected one another. Small timing delays, noisy sensor values, unstable controller parameters, unclear state transitions, or network configuration issues could cause the car to lose the line, falsely detect obstacles, display inaccurate movement, or fail to publish status. This project showed that autonomous navigation requires not only correct code, but also careful sensor testing, parameter tuning, filtering, state-machine design, and repeated real-world debugging.